

AssureEver — API Backend

API REST FastAPI pour la plateforme de gestion de la sécurité sociale AssureEver.

Dépôt frontend : [assureever-frontend](#)

Stack technique

Composant	Technologie
Framework	FastAPI
ORM	SQLAlchemy 2.0 (async)
Migrations	Alembic
Base de données	PostgreSQL 16
Validation	Pydantic v2
Auth	JWT compatible NextAuth.js v5
Hachage mdp	passlib + bcrypt
PDF	ReportLab
Déploiement	Render (API) + Neon (BD)

Architecture

```
app/  
├─ core/  
│   ├── config.py           → Variables d'environnement (pydantic-settings)  
│   ├── security.py         → Vérification JWT NextAuth + dépendances de rôles  
│   └─ exceptions.py       → Handlers d'erreurs globaux  
├─ models/                 → Modèles SQLAlchemy (tables PostgreSQL)  
├─ schemas/               → Schémas Pydantic (validation requêtes/réponses)  
└─ services/              → Logique métier et règles du cahier d'analyse
```

```
└─ routers/           → Endpoints REST par module
└─ main.py            → Point d'entrée FastAPI
```

Cas d'utilisation implémentés

UC	Acteur	Description	Statut
UC0	Assureur	Enregistrer un médecin (généraliste/spécialiste)	✓ Ajouté (absent du cahier original — voir note)
UC1	Assureur	Inscrire un assuré	✓
UC2	Assureur	Enregistrer un médecin traitant pour un assuré	✓
UC3	Assureur	Effectuer un remboursement	✓
UC4	Assureur	Imprimer / télécharger une facture PDF	✓
UC5	Assureur	Compléter une feuille de maladie	✓
UC6	Médecin	Enregistrer une feuille de maladie	✓
UC7	Médecin	Prescrire un médicament	✓
UC8	Médecin	Prescrire une consultation chez un spécialiste	✓
UC9	Assureur + Médecin	S'authentifier	✓

Note — UC0 (Enregistrer un médecin) : Ce cas d'utilisation est **absent du cahier d'analyse original** (lacune de conception identifiée lors de l'analyse). Il a été ajouté car :

- UC8 pose comme pré-condition que "*le spécialiste existe dans le système*".
- UC2 requiert que le médecin traitant soit trouvable dans le système.
- Le package *Inscriptions* du cahier décrit explicitement "*l'enregistrement des médecins au sein de l'organisme de sécurité sociale*".

C'est donc l'Assureur qui inscrit les médecins (généralistes et spécialistes), exactement comme il inscrit les assurés. Un médecin ne s'inscrit pas lui-même. L'enregistrement crée

simultanément la fiche médecin et un compte utilisateur pour permettre au médecin de s'authentifier (UC9).

Endpoints REST

Médecins (/api/v1/medecins) — UC0

Méthode	Endpoint	Rôle	Description
GET	/	Tous	Lister les médecins (filtre type + recherche)
POST	/	ASSUREUR	Enregistrer un médecin + créer son compte utilisateur
GET	/generalistes	ASSUREUR	Lister uniquement les généralistes (pour UC2)
GET	/specialistes	Tous	Lister les spécialistes (filtre spécialité, pour UC8)
GET	/{"id"}	Tous	Fiche complète d'un médecin
PATCH	/{"id"}	ASSUREUR	Mettre à jour téléphone ou spécialité
DELETE	/{"id"}	ASSUREUR	Désactiver le compte (données conservées)

Corps de la requête POST :

```
{
  "matricule": "MED-2024-001",
  "nom": "Kenne",
  "prenom": "Diha",
  "type_medecin": "GENERALISTE",
  "specialite": null,
  "telephone": "+237 6XX XXX XXX",
  "email": "kenne.diha@clinique.cm",
  "password": "motdepasse123"
}
```

Pour un spécialiste, type_medecin = "SPECIALISTE" et specialite est **obligatoire** (ex : "Cardiologie", "Ophtalmologie").

Assurés (`/api/v1/assures`) — UC1, UC2 — Rôle : ASSUREUR

Méthode	Endpoint	Description
GET	<code>/</code>	Lister (pagination + recherche)
POST	<code>/</code>	Inscrire un assuré (contrôle doublon)
GET	<code>/{{id}}</code>	Fiche complète d'un assuré
PUT	<code>/{{id}}/medecin-traitant</code>	Associer un médecin traitant (généraliste uniquement)

Feuilles de maladie (`/api/v1/feuilles-maladie`) — UC5, UC6

Méthode	Endpoint	Rôle	Description
GET	<code>/assure/{{id}}</code>	Tous	Toutes les feuilles d'un assuré
GET	<code>/assure/{{id}}/en-attente</code>	ASSUREUR	Feuilles à rembourser
POST	<code>/</code>	MEDECIN	Enregistrer une feuille
PATCH	<code>/{{id}}</code>	ASSUREUR	Compléter une feuille

Prescriptions (`/api/v1/prescriptions`) — UC7, UC8 — Rôle : MEDECIN

Méthode	Endpoint	Description
POST	<code>/medicament</code>	Prescrire un médicament
POST	<code>/consultation-specialiste</code>	Prescrire une consultation chez un spécialiste

Remboursements (`/api/v1/remboursements`) — UC3, UC4 — Rôle : ASSUREUR

Méthode	Endpoint	Description
GET	<code>/assure/{{id}}</code>	Historique des remboursements
POST	<code>/</code>	Effectuer un remboursement
GET	<code>/{{id}}/facture</code>	Télécharger la facture PDF

Règles métier

Règle	Description
Inscription médecin	Seul l' Assureur peut enregistrer un médecin (UC0)
Médecin traitant	Seul un médecin GENERALISTE peut être désigné médecin traitant (UC2)
Taux remboursement	100% pour un généraliste, 80% pour un spécialiste (UC3)
Spécialité obligatoire	Un médecin SPECIALISTE doit avoir une spécialité renseignée (UC0)
Prescription spécialiste	Le médecin cible d'une prescription doit être de type SPECIALISTE (UC8)
Contrôle doublon assuré	Vérification nom + prénom + date de naissance (UC1)
Contrôle doublon médecin	Vérification sur le matricule et l'email (UC0)
Statuts feuille	EN_ATTENTE → COMPLETE → REMBOURSEE
Désactivation médecin	Le compte est désactivé mais les données historiques sont conservées

Installation locale

Prérequis

- Python 3.12+
- PostgreSQL 16+

```
# Cloner
git clone https://github.com/ALEMDJOU/assureever-api.git
cd assureever-api

# Environnement virtuel
python -m venv venv
source venv/bin/activate # Windows : venv\Scripts\activate

# Dépendances
```

```
pip install -r requirements.txt

# Variables d'environnement
cp .env.example .env
# Éditer .env

# Migrations
alembic upgrade head

# Démarrer
uvicorn app.main:app --reload
```

Via Docker Compose

```
docker-compose up -d
```

Documentation interactive

Interface	URL
Swagger UI	http://localhost:8000/docs
ReDoc	http://localhost:8000/redoc
Health check	http://localhost:8000/health

Variables d'environnement

```
DATABASE_URL=postgresql+asyncpg://user:password@host:5432/securite_sociale
NEXTAUTH_SECRET=votre-secret-32-caracteres-minimum
FRONTEND_URL=http://localhost:3000
ENVIRONMENT=development
```

Migrations Alembic

```
# Créer une migration
alembic revision --autogenerate -m "description"

# Appliquer
alembic upgrade head

# Revenir en arrière
alembic downgrade -1
```

Déploiement (Render + Neon)

1. Créer une base de données PostgreSQL sur **Neon**.
2. Connecter ce dépôt à **Render** (Web Service).
3. Configurer les variables d'environnement sur Render (dont `DATABASE_URL` pointant vers Neon).
4. Render détecte le `Dockerfile` automatiquement et effectue le build.
5. Exécuter les migrations (`alembic upgrade head`) en local (pointant vers Neon) ou via le shell Render.

Auteurs

Projet tutoré 3GI — Conception des Systèmes d'Information Supervisé par Dr. Anne Marie CHANA et Dr. Jaures Styve KAMENI